

Aula 4: Indexação e busca em sequências

Luís Felipe

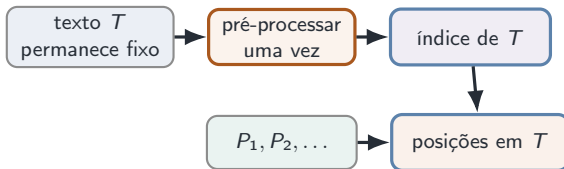
UFF

25 de Agosto de 2026

Um texto fixo pode receber muitas buscas diferentes

Situação recorrente

Temos um **texto fixo** T , por exemplo um genoma de referência, e recebemos **padrões** $P_1, P_2, \dots$. Para cada padrão, queremos localizar suas ocorrências.



O papel de um índice: Gastar tempo e espaço antecipadamente para evitar uma nova varredura completa de T a cada consulta.

Buscar exatamente é devolver as posições do padrão

Busca exata

Dados $T \in \Sigma^n$ e $P \in \Sigma^m$, determinar

$$\text{Occ}(P, T) = \{i \in \{0, \dots, n - m\} : T[i..i + m - 1] = P\}.$$

Cada i é a posição inicial de uma cópia exata de P em T .

	0	1	2	3	4	5	6	7
$T =$	A	C	G	A	C	G	T	A
$P =$	A	C	G					

$\text{Occ}(P, T) = \{0, 3\}$

Três saídas possíveis

Decisão: ocorre?

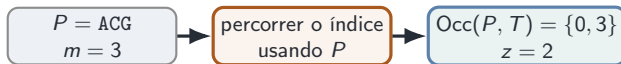
Contagem: quantas vezes?

Enumeração: em quais posições?

O custo da consulta inclui ler P e escrever a resposta

Consulta

O índice de T já está pronto. Recebemos um novo padrão P e pedimos $\text{Occ}(P, T)$. Denote $m = |P|$ e $z = |\text{Occ}(P, T)|$.



- os m símbolos de P determinam o caminho procurado no índice;
- se a consulta enumera z posições, apenas escrevê-las custa $\Omega(z)$.

Meta natural

$O(m + z)$: tempo proporcional ao que entrou na consulta mais o que precisou sair. Esse limite não inclui o pré-processamento, realizado anteriormente.

O índice compensa quando o mesmo texto recebe muitas consultas

Suponha r padrões, todos com comprimento m , e respostas de tamanhos $z_1, \dots, z_r$.

Sem índice

Para cada padrão, percorremos as n posições de T :

$$O\left(rn + rm + \sum_j z_j\right).$$

rn : varreduras; rm : leitura dos padrões;
 $\sum_j z_j$: saída. Para $n \gg m$, domina rn .

Exemplo: Se $n = 10^9$, $m = 20$ e $r = 10^6$, varrer T para cada padrão examina cerca de 10^{15} posições; indexar separa o custo único em T do custo de cada consulta.

Com árvore de sufixos

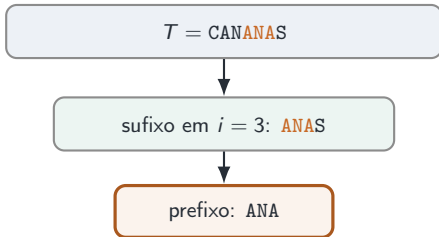
Construímos o índice uma vez e consultamos depois:

$$O\left(n + rm + \sum_j z_j\right).$$

n : índice; rm : consultas; $\sum_j z_j$: saída. O termo rn desaparece.

Todo substring é prefixo de algum sufixo

Se $X = T[i..j]$, então X é exatamente o prefixo de comprimento $j - i + 1$ do sufixo $T[i..n - 1]$.



O ganho conceitual

Em vez de indexar separadamente $O(n^2)$ intervalos possíveis, **organizamos os n sufixos**; todos os **substrings aparecem como prefixos de caminhos desses sufixos**.

Cuidado com o espaço

Há apenas n sufixos, mas copiá-los integralmente ainda usa $n + (n - 1) + \dots + 1 = \Theta(n^2)$ caracteres. A economia surge ao **compartilhar prefixos** em uma **trie** e depois **compactar caminhos** (**árvore de sufixo**).

Uma trie compartilha os prefixos das palavras inseridas

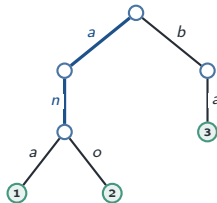
Trie:

É uma árvore em que cada aresta armazena um símbolo e **cada caminho desde a raiz soletra um prefixo**.

O nome **trie** foi formado a partir de *retrieval* (Muitos autores pronunciam “try” para distingui-la de *tree*).

Inserir uma palavra x

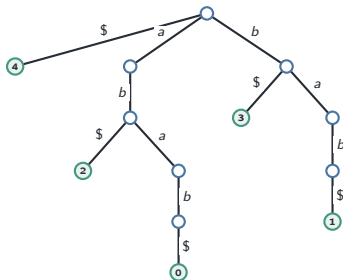
1. comece na raiz;
2. para cada símbolo $x[j]$, siga a aresta rotulada $x[j]$;
3. se a aresta não existir, crie-a;
4. marque o nó final como término de palavra.



Palavras ana, ano e ba. As duas primeiras compartilham o prefixo an, desenhado em azul.

²⁵A trie de sufixos insere todos os sufixos de T

Para $T = abab$, inserimos $abab\$, bab\$, ab\$, b\$$ e $\$$. A folha guarda a posição inicial.



O símbolo \$ é um **terminador exclusivo** ($T\$$): ele não pertence ao alfabeto de T e ocorre somente no final.

Assim, nenhum sufixo de T_S é prefixo de outro e cada sufixo termina em uma folha própria.

Por exemplo, sem \$, ab seria prefixo de abab; com o terminador, ab\$ e abab\$ seguem caminhos diferentes.

Se $N = |T\$|$, os sufixos têm comprimentos $N, N-1, \dots, 1$. Assim, a inserção ingênua lê essa soma de símbolos, portanto custa $\Theta(N^2)$ tempo.

Cada aresta da trie guarda um símbolo; o número de arestas é $\Theta(N^2)$ no pior caso.

Compactar caminhos produz a árvore de sufixos

Regra de compactação

Todo caminho interno cujos nós têm **um único filho** é substituído por uma aresta rotulada pela **concatenação dos símbolos do caminho**.



Rótulos ocupam $O(1)$ espaço

Uma aresta não precisa copiar *ana*. Ela guarda um par (ℓ, r) que aponta para $T[\ell..r]$. O texto original permanece disponível para comparações.

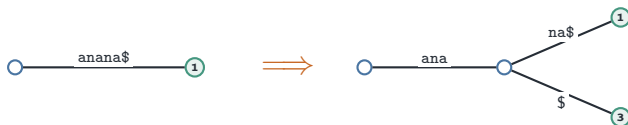
A **estrutura preserva** os mesmos caminhos e folhas da trie de sufixos, mas sem materializar cada caractere de um caminho não ramificado em um nó separado.

Inserir um sufixo pode exigir dividir uma aresta

Considere os sufixos de banana\$ iniciados nas posições 1 e 3:

$$T[1..] = \text{anana\$} \quad \text{e} \quad T[3..] = \text{ana\$}.$$

Suponha que anana\$ já tenha sido inserido e que desejemos inserir ana\$.



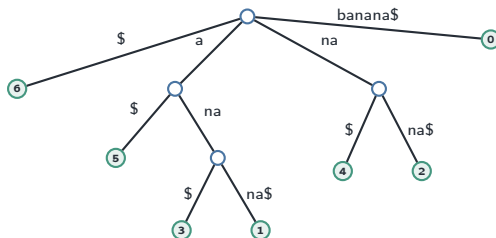
1. compare o novo sufixo com a aresta existente;
2. identifique o prefixo comum ana;
3. divida a aresta e crie um nó após esse prefixo;
4. mantenha na\$ para a folha 1 e crie \$ para a folha 3.

Complexidade

A inserção ingênua dos sufixos pode custar $\Theta(N^2)$. Para alfabetos de tamanho constante, como o do DNA, algoritmos como os de Ukkonen e de McCreight constroem a árvore em $O(N)$.

Exemplo

Árvore de sufixo de banana:

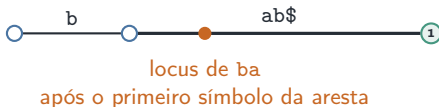


- o caminho raiz-folha i soletra o sufixo iniciado em i ;
- o nó alcançado por *ana* possui as folhas 1 e 3;
- portanto, *ana* ocorre nas posições 1 e 3.

Um substring pode terminar em nó ou dentro de uma aresta

Locus de uma palavra X

É o ponto alcançado após consumir todos os símbolos de X desde a raiz. Ele pode ser um **nó explícito** ou um **ponto implícito** dentro de uma aresta.



Consequência

Não é necessário existir um nó para cada substring. A árvore representa também os substrings que terminam no interior de rótulos compactados.

A compactação reduz a estrutura para espaço linear

Sejam $N = |T\$|$, L o número de folhas, I o número de nós internos e E o número de arestas.

Duas formas de contar as arestas

Cada nó, exceto a raiz, recebe uma aresta (**pois cada nó diferente da raiz possui um pai**):

$$E = L + I - 1.$$

Na árvore compactada, todo nó interno tem pelo menos dois filhos:

$$E \geq 2I.$$

Assim, $L + I - 1 \geq 2I$ e, portanto, $I \leq L - 1$.

Como existe uma folha por sufixo, $L = N$. Logo,

$$I \leq N - 1, \quad E \leq 2N - 2.$$

Consequência

Cada aresta guarda dois índices (ℓ, r) do texto, e não uma cópia de $T[\ell..r]$. Assim, a topologia e os rótulos ocupam $O(N)$ espaço.

Consultar a árvore consome o padrão uma única vez

1. **comece na raiz** com $j = 0$;
2. **escolha a aresta** cujo primeiro símbolo é $P[j]$;
3. **compare P com o rótulo da aresta** até terminar o rótulo, terminar P ou encontrar desacordo;
4. se P terminar, **liste as folhas abaixo do locus**; se houver desacordo, a resposta é vazia.

Exemplo em banana\$

Para $P = \text{ana}$, siga a aresta a e depois na . O padrão termina no nó de profundidade de string 3; as folhas abaixo são 1 e 3.

Complexidade $O(m + z)$

Cada símbolo de P é comparado no máximo uma vez ao longo do caminho: $O(m)$. Depois percorremos exatamente a subárvore necessária para emitir suas z folhas: $O(z)$. Supomos acesso $O(1)$ à aresta pelo primeiro símbolo, como em um alfabeto constante.

WOTD constrói a árvore de sufixos top-down

Objetivo

Construir diretamente a árvore de sufixos compactada, sem precisar construir antes a trie completa de todos os sufixos.

WOTD: *Write-Only Top-Down*

Em cada etapa, considere os sufixos associados ao nó atual:

1. agrupe os sufixos pelo **primeiro símbolo** ainda não processado;
2. para cada grupo, determine o **maior prefixo comum**;
3. **crie uma aresta** rotulada por esse prefixo;
4. **remova o prefixo** utilizado e repita o procedimento recursivamente;
5. quando um grupo contém apenas um sufixo, **crie uma folha** identificada pela sua posição original.

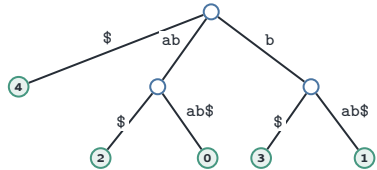
Por que a árvore já é construída compactada?

Os sufixos abab\$ e ab\$ começam com a, mas seu maior prefixo comum é ab. Assim, criamos diretamente uma aresta rotulada por ab.

A construção começa na raiz e continua recursivamente em cada subárvore.

Exemplo: WOTD para o texto abab\$

Grupo	Sufixos	Prefixo comum	Resultado
a	abab\$, ab\$	ab	nó interno
b	bab\$, b\$	b	nó interno
\$	\$	\$	folha 4



Recursão: remover o prefixo utilizado na aresta

0: abab\$ = ab · ab\$ 1: bab\$ = b · ab\$

2: ab\$ = ab · \$ 3: b\$ = b · \$

Após remover os prefixos ab e b, restam ab\$ e \$ em cada grupo.

Como esses trechos começam com símbolos diferentes, são criadas duas arestas, cada uma levando à folha correspondente.

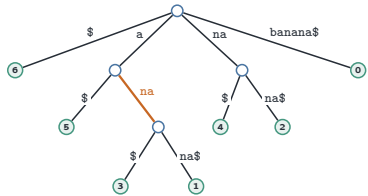
Comparação entre estratégias de construção

Estratégia	Tempo	Espaço
Construir a trie e depois compactar	$\Theta(n^2)$	$\Theta(n^2)$
WOTD: construir diretamente	$O(n^2)$	$O(n)$
Ukkonen: construção direta	$O(n)$	$O(n)$

Para Ukkonen, considera-se um alfabeto de tamanho constante, como o alfabeto do DNA.

Exemplo: WOTD para o texto banana\$

Grupo	Sufixos	Prefixo comum	Resultado
\$	\$	\$	folha 6
a	anana\$, ana\$, a\$	a	nó interno
b	banana\$	banana\$	folha 0
n	nana\$, na\$	na	nó interno



Recursão: alguns sufixos continuam agrupados

No grupo iniciado em a, removemos o prefixo comum:

$$\begin{aligned}
 1 : \text{anana\$} &= a \cdot \text{nana\$}, \\
 3 : \text{ana\$} &= a \cdot \text{na\$}, \\
 5 : \text{a\$} &= a \cdot \$.
 \end{aligned}$$

O sufixo 5 segue pela aresta \$. Os sufixos 1 e 3 continuam juntos: seus trechos restantes compartilham na, produzindo a aresta destacada.

Depois de remover na, restam na\$ e \$, que levam às folhas 1 e 3.

Aplicações: Comprimento e frequência

Profundidade de string

A profundidade de string de um ponto é o **número total de caracteres** nos rótulos da raiz até ele.

Maior substring repetido

Queremos maximizar o **comprimento**.

Um substring ocorre ao menos duas vezes quando seu **locus possui ao menos duas folhas descendentes**.

Portanto, procuramos o nó interno de maior profundidade de string.

Em banana, o nó ana tem profundidade 3 e folhas 1, 3. Resposta: ana.

Substring mais frequente

Queremos maximizar o **número de ocorrências**. Para um locus, esse número é a quantidade de folhas descendentes.

Em banana, a possui três folhas e ocorre três vezes, embora seja menor que ana.

Obs.: “Maior” refere-se ao comprimento; “mais frequente”, à contagem de folhas.

Como construir uma árvore de sufixos generalizada?

Passo 1: separar as sequências com terminadores

Considere:

$$S = \text{GATTCGTA} \quad \text{e} \quad T = \text{CCTCGTG}.$$

Utilizamos dois símbolos distintos que não pertencem ao alfabeto:

$$U = S\#T\$ = \text{GATTCGTA}\#\text{CCTCGTG}\$.$$

Sufixos associados a S

S_0	GATTCGTA#
S_1	ATTCGTA#
S_2	TTCGTA#
S_3	TCGTA#
S_4	CGTA#
S_5	GTA#
S_6	TA#
S_7	A#

Sufixos associados a T

T_0	CCTCGTG\$
T_1	CTCGTG\$
T_2	TCGTG\$
T_3	CGTG\$
T_4	GTG\$
T_5	TG\$
T_6	G\$

Passo 2: inserir todos os sufixos na mesma árvore

Os sufixos são **agrupados por seus prefixos comuns**, como na árvore de uma única string.

Cada folha **registra o sufixo** se veio de S ou de T .

Uma árvore generalizada compara duas sequências

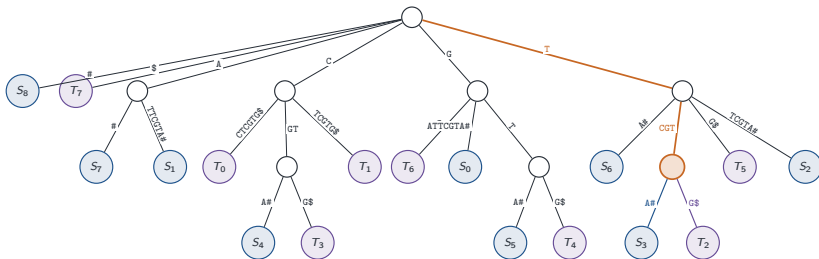
$S = \text{GATTCGTA}$

$T = \text{CCTCGTG}$

Construção

Organizamos os sufixos de $\text{GATTCGTA}\#$ e $\text{CCTCGTG}\$$.

A folha S_i ou T_j indica a **sequência** e a **posição inicial** do sufixo correspondente.



Substring comum

O caminho destacado soletra $T \cdot \text{CGT}$ e sua subárvore contém as folhas S_3 e T_2 .

Assim, TCGT ocorre nas duas sequências.

Um MUM é comum, único nas duas sequências e maximal

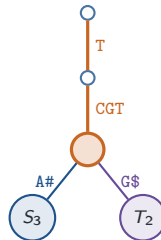
$S = \text{GATTCGTA}$

$T = \text{CCTCGTG}$

MUM: *maximal unique match*

Uma palavra X deve:

1. ocorrer nas duas sequências;
2. ocorrer uma única vez em cada uma;
3. não admitir extensão comum à esquerda nem à direita.



O nó de TCGT possui exatamente as folhas S_3 e T_2 . Logo, existe uma ocorrência em cada sequência.

Por que isso é útil?

MUMs são âncoras exatas não ambíguas para alinhar grandes sequências antes de tratar diferenças entre elas.

A árvore permite testar as três condições de um MUM

Considere o locus de $X = \text{TCGT}$ na árvore generalizada.

1. **Comum**: abaixo do locus há folhas das duas cores.
2. **Único em cada sequência**: abaixo do locus há exatamente duas folhas, uma de S e uma de T . Se houvesse duas folhas da mesma cor, X apareceria mais de uma vez naquela sequência.
3. **Maximal à esquerda**: os símbolos anteriores às duas ocorrências são diferentes, ou uma ocorrência começa na posição 0. No exemplo, $S[2] = T \neq C = T[1]$.

O vetor de sufixos guarda a ordem sem guardar a árvore

Definição

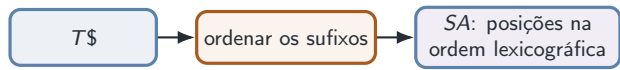
O **suffix array** $SA[0..N - 1]$ é uma permutação das posições de T tal que

$$T[SA[0]..] < T[SA[1]..] < \dots < T[SA[N - 1]..]$$

na ordem lexicográfica.

Objetivo

Representar a ordem dos sufixos em um vetor de N inteiros. Isso **reduz constantes de espaço** e melhora a localidade de memória em comparação com uma árvore cheia de ponteiros.



Construção direta: liste e ordene os sufixos

Para $T = \text{banana\$}$:

i	sufixo	ordem	$SA[r]$	sufixo ordenado
0	banana\$	0	6	\$
1	anana\$	1	5	a\$
2	nana\$	2	3	ana\$
3	ana\$	3	1	anana\$
4	na\$	4	0	banana\$
5	a\$	5	4	na\$
6	\$	6	2	nana\$

$$SA = [6, 5, 3, 1, 0, 4, 2].$$

Como interpretar

$SA[2] = 3$ significa que o terceiro sufixo na ordem lexicográfica (índice 2) começa na posição 3 e é ana\$.

Como os sufixos estão ordenados, aqueles iniciados por um mesmo padrão formam um intervalo consecutivo, que pode ser localizado por busca binária.

Para $P = \text{ana}$: $SA[2..3] = [3, 1]$.

Portanto, P ocorre nas posições 3 e 1 do texto.

Comparar sufixos inteiros pode tornar a construção cara

Construção direta

Os N sufixos contêm, ao todo,

$$N + (N - 1) + \cdots + 1 = \Theta(N^2)$$

caracteres. Uma comparação lexicográfica pode examinar $\Theta(N)$ símbolos; portanto, uma ordenação ingênua não fornece uma construção eficiente garantida:

$$\underbrace{O(N \log N)}_{\text{comparações entre sufixos}} \cdot \underbrace{O(N)}_{\text{caracteres por comparação}} = O(N^2 \log N).$$

Ideia do *prefix doubling*

Em vez de comparar palavras longas, atribuímos **ranks inteiros** aos prefixos:

- primeiro, ordenamos prefixos de comprimento 1;
- se conhecemos o rank dos blocos de comprimento h , o prefixo de comprimento $2h$ é descrito pelo par de ranks de suas duas metades;
- ordenamos esses pares e repetimos até $h \geq N$.

Objetivo da técnica

Substituir comparações de strings por ordenações de pares de inteiros.

Primeira rodada: ranks dos símbolos e pares de dois símbolos

Para banana\$, os ranks iniciais são:

$$\$ \mapsto 0, \quad a \mapsto 1, \quad b \mapsto 2, \quad n \mapsto 3.$$

Para $h = 1$, a chave da posição i é:

$$(\text{rank}[i], \text{rank}[i + 1]).$$

Fora do texto, utilizamos -1 .

i	Sufixo	Par inicial	Ordem do par	Novo rank
6	<u>\$</u> _	$(0, -1)$	1	0
5	<u>a</u> <u>\$</u>	$(1, 0)$	2	1
1	<u>a</u> <u>n</u> ana\$	$(1, 3)$	3	2
3	<u>a</u> <u>n</u> a\$	$(1, 3)$	3	2
0	<u>b</u> <u>a</u> nana\$	$(2, 1)$	4	3
2	<u>n</u> <u>a</u> na\$	$(3, 1)$	5	4
4	<u>n</u> <u>a</u> \$	$(3, 1)$	5	4

Pelo novo rank podemos ter uma nova ordem dos dois primeiros elementos:

\$, a\$, an, ba, na.

O que significa um empate?

As posições 1 e 3 recebem o mesmo novo rank porque começam pelo mesmo bloco an.

Novos ranks para os blocos de dois símbolos

$$\$ < a\$ < an < ba < na.$$

Bloco	\$	a\$	an	ba	na
Novo rank	0	1	2	3	4

Segunda rodada: dois ranks descrevem quatro símbolos

Agora cada rank resume os primeiros $h = 2$ símbolos. A chave para quatro símbolos é $(\text{rank}_2[i], \text{rank}_2[i + 2])$.

i	sufixo	par de ranks	ordem final
6	<u>\$</u> _	$(0, -1)$	0
5	<u>a</u> \$ _	$(1, -1)$	1
3	<u>an</u> <u>a</u> \$	$(2, 1)$	2
1	<u>an</u> <u>ana</u> \$	$(2, 2)$	3
0	<u>ba</u> <u>nana</u> \$	$(3, 4)$	4
4	<u>na</u> _ \$	$(4, 0)$	5
2	<u>na</u> <u>na</u> \$	$(4, 4)$	6

Todos os pares são distintos, logo $SA = [6, 5, 3, 1, 0, 4, 2]$.

Complexidade

Há $\lceil \log_2 N \rceil$ rodadas. Com ordenação linear de pares de ranks inteiros, cada rodada custa $O(N)$: total de tempo $O(N \log N)$ e de espaço $O(N)$.

Sufixos iniciados por P formam um intervalo contíguo

Na lista ordenada de banana\$, tome $P = \text{ana}$:

r	$SA[r]$	sufixo
0	6	\$
1	5	a\$
2	3	ana\$
3	1	anana\$
4	0	banana\$
5	4	na\$
6	2	nana\$

Por que o intervalo é contíguo?

Todas as palavras que começam por P ficam juntas na ordem lexicográfica:

- Antes delas aparecem sufixos menores que P ;
- Depois, sufixos que divergem para um símbolo maior.
- Um sufixo sem prefixo P não pode ficar entre dois que o possuem.

Busca

Duas buscas binárias encontram os limites esquerdo e direito do intervalo.

A primeira busca binária encontra o início do intervalo

Para $P = \text{ana}$, procure o primeiro sufixo que não é lexicograficamente menor que P . Use o intervalo semiaberto $[lo, hi)$.

lo	hi	mid	sufixo em $SA[mid]$	decisão
0	7	3	anana\$ começa por P	$hi \leftarrow 3$
0	3	1	a\$ < P	$lo \leftarrow 2$
2	3	2	ana\$ começa por P	$hi \leftarrow 2$

Ao terminar, $lo = hi = 2$. Portanto, o primeiro sufixo iniciado por P está no rank $L = 2$.

Custo

Há $O(\log N)$ comparações. Se cada comparação lê até m símbolos de P , o limite simples é $O(m \log N)$.

A segunda busca encontra o fim e o intervalo devolve as posições

Agora procure o primeiro sufixo estritamente depois do bloco que começa por P .

lo	hi	mid	sufixo em $SA[mid]$	decisão
0	7	3	anana\$ começa por P	$lo \leftarrow 4$
4	7	5	na\$ > P	$hi \leftarrow 5$
4	5	4	banana\$ > P	$hi \leftarrow 4$

Logo, $R = 4$ e o intervalo é $SA[2..3] = [3, 1]$.

Resposta

Os valores do vetor são as posições no texto:

$$\text{Occ}(\text{ana}, \text{banana}) = \{3, 1\} = \{1, 3\}.$$

Listar os $z = R - L$ valores acrescenta $O(z)$ ao tempo da consulta.

Complexidade total simples

$O(m \log N + z)$ com apenas o suffix array. Há formas de evitar comparações repetidas de prefixos.

A BWT pode ser construída por sufixos ou rotações

A **transformação de Burrows–Wheeler**, ou **BWT**, reorganiza os símbolos de um texto de maneira reversível, favorecendo sua compressão e a construção de índices para busca de padrões. Considere o texto

$$T = \text{ATATAT}, \quad X = \text{ATATAT\$}, \quad N = |X|.$$

O terminador \$ ocorre apenas uma vez e é menor que os demais símbolos.

Pelos sufixos ordenados

Ordene os sufixos e registre o símbolo anterior ao início de cada um:

$$L[r] = X[(SA[r] - 1) \bmod N].$$

r	$SA[r]$	Sufixo	$L[r]$
0	6	\$	T
1	4	AT\$	T
2	2	ATAT\$	T
3	0	ATATAT\$	\$
4	5	T\$	A
5	3	TAT\$	A
6	1	TATAT\$	A

Pelas rotações ordenadas

Ordene as rotações cíclicas de X e registre o último símbolo de cada uma.

r	Rotação	Último símbolo
0	\$ATATAT	T
1	AT\$ATAT	T
2	ATAT\$AT	T
3	ATATAT\$	\$
4	T\$ATATA	A
5	TAT\$ATA	A
6	TATAT\$A	A

O terminador único garante que a ordem dos sufixos coincide com a ordem das rotações correspondentes. Assim: $SA = [6, 4, 2, 0, 5, 3, 1] \implies L = \text{TTT$AAA}$.

O suffix array armazena **posições**; a BWT armazena **símbolos**.

BWT favorece compressão e índices de busca compactos

Os mesmos caracteres, reorganizados

Texto original: A T A T A T \$
BWT: T T T \$ A A A

No texto original, os símbolos alternam.

Na BWT, aparecem agrupados em três blocos:

TTT, \$, AAA.

Por que esses blocos ajudam?

Blocos consecutivos de símbolos iguais, chamados **runs**, podem ser representados por símbolo e quantidade: (T, 3), (\$, 1), (A, 3).

Em textos grandes e repetitivos, esse agrupamento favorece a compressão.

Diferença entre suffix array e BWT

O **suffix array** armazena posições ordenadas e permite localizar ocorrências por busca binária.

A **BWT** armazena caracteres reorganizados. Com contagens auxiliares e posições amostradas, ela permite construir índices compactos, como o FM-index, sem guardar o suffix array completo.

Por que símbolos iguais podem ficar agrupados?

A ordem lexicográfica aproxima sufixos que começam com prefixos semelhantes.
A BWT registra o símbolo anterior a cada um desses sufixos.
Quando contextos semelhantes são precedidos pelo mesmo símbolo, esses símbolos ficam próximos e podem formar **runs**.

O agrupamento não é garantido

$$\text{BWT}(\text{ATATAT\$}) = \text{TTT\$AAA},$$

mas:

$$\text{BWT}(\text{AAATTT\$}) = \text{T\$AATTA}.$$

A BWT favorece compressão em muitos textos repetitivos, mas não necessariamente diminui o número de blocos em toda entrada.

Como reconstruir o texto a partir da BWT?

Considere a BWT: $L = \text{TTT}\$AAA$, com a ordem lexicográfica: $\$ < A < T$.

Primeira rodada

Ordenamos os símbolos de L : $\$, A, A, A, T, T, T$.

Segunda rodada

Em cada linha i , colocamos $L[i]$ antes da string obtida na rodada anterior. Depois, ordenamos os resultados.

$L[i]$	Rodada 1	Antes da ordenação	Depois da ordenação
T	\$	T\$	\$A
T	A	TA	AT
T	A	TA	AT
\$	A	\$A	AT
A	T	AT	T\$
A	T	AT	TA
A	T	AT	TA

O que permanece fixo?

Os símbolos utilizados em cada rodada continuam sendo, nessa ordem, T, T, T, \$, A, A, A.

As strings produzidas é que são novamente ordenadas.

A repetição do procedimento recupera o texto original

Em cada rodada

Colocamos novamente os símbolos de $L = TTT\$AAA$ à esquerda das linhas atuais e ordenamos lexicograficamente as strings resultantes.

Rodada 2	Rodada 3	Rodada 7
\$A	\$AT	\$ATATAT
AT	AT\$	AT\$ATAT
AT	ATA	ATAT\$AT
AT	ATA	ATATAT\$
T\$	T\$A	T\$ATATA
TA	TAT	TAT\$ATA
TA	TAT	TATAT\$A

Por que aparecem linhas repetidas?

Nas primeiras rodadas, algumas strings ainda compartilham o mesmo prefixo. À medida que acrescentamos novos símbolos, essas linhas passam a ser distinguidas.

Recuperação do texto

A única linha terminada em \$ é: $X = ATATAT\$$.
Removendo o terminador, recuperamos: $T = ATATAT$.

Referências principais

- D. Gusfield. *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.
- M. Crochemore, C. Hancart e T. Lecroq. *Algorithms on Strings*. Cambridge University Press, 2007.
- R. Giegerich, S. Kurtz e J. Stoye. Efficient Implementation of Lazy Suffix Trees. *Software: Practice and Experience*, 2003.
- U. Manber e G. Myers. Suffix Arrays: A New Method for On-Line String Searches. *SIAM Journal on Computing*, 1993.
- T. Kasai et al. Linear-Time Longest-Common-Prefix Computation in Suffix Arrays. CPM, 2001.
- M. Burrows e D. Wheeler. *A Block-sorting Lossless Data Compression Algorithm*. DEC, 1994.
- P. Ferragina e G. Manzini. Opportunistic Data Structures with Applications. FOCS, 2000.
- A. Delcher et al. Alignment of Whole Genomes. *Nucleic Acids Research*, 1999.
- S. Altschul et al. Basic Local Alignment Search Tool. *Journal of Molecular Biology*, 1990.
- H. Li e R. Durbin. Fast and Accurate Short Read Alignment with Burrows–Wheeler Transform. *Bioinformatics*, 2009.